

Exercícios - GPIO

A entrega deste exercício deve ser feita via Teams

1. Análise de Código

1) Dado o trecho de código abaixo, explique com suas palavras o que ele faz, sequencialmente:

```
#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>

int main(void)
{
    DDRC |= (1 << PC5);

    while (1)
    {
        PORTC ^= (1 << PC5);
        _delay_ms(500);

        while(!PINC & (1 << PC6)){
            PORTC &= ~(1 << PC5);
        }

        return 0;
    }
}
```

Caso julgue necessário, execute este código e observe seu comportamento na placa.

2) O código a seguir faz uma tarefa simples parecer complexa. Além disso, ele “gasta” memória e processamento a mais em algumas abordagens. Analise e reescreva o código para ser o mais otimizado possível, e ainda assim realizar seu objetivo final:

```
uint8_t buttons = 0;
DDRB |= (1<<PB0) + (1<<PB1) + (1<<PB2);
DDRD &= ~(1<<PD0) + (1<<PD1) + (1<<PD2));

while (1)
{
    buttons = PIND;
    switch(buttons)
    {
        case 0:
            PORTB &= ~ (255);
            break;

        case 1:
            PORTB |= 1;
            break;

        case 2:
            PORTB |= 2;
            break;

        case 3:
            PORTB |= 3;
            break;
    }
}
```

2. Construção de códigos

1) Crie um programa que possua 5 pinos de entrada no Portal B, 1 LED1 como saída no Portal D, e um LED2 como saída no Portal C. O programa deve funcionar como um identificador de padrões, onde o LED1 deve permanecer aceso e o LED2 apagado, enquanto as entradas possuírem bits intercalados. Se todos as entradas estiverem com nível lógico alto ou baixo, o LED2 deve permanecer aceso, e o LED1 apagado.

Exemplo:

ENTRADAS= 10101, LED1 ACESO

ENTRADAS= 10010, NENHUM LED ACESO

ENTRADAS= 01010, LED1 ACESO

ENTRADAS= 00000, LED2 ACESO

ENTRADAS= 11111, LED2 ACESO

2) Crie um programa em C que funcione como uma fechadura eletrônica. O programa deve possuir 4 botões de entrada, com resistores de pull-up internos. Os 4 botões devem ser conectados aos pinos PB1, PB2, PB3 e PB4. O comando que libera a trava do sistema deve ser conectado ao pino PD3.

O sistema só será liberado se os botões forem pressionados em uma determinada ordem: PB2-> PB3-> PB4-> PB1. Sendo que, nenhum pino pode ser pressionado e solto, ou seja, ao final da sequência todos pinos devem estar pressionados para validar o acionamento, que será indicado pelo nível lógico alto no pino PD3.

3. Para refletir - Conceitos e Teoria

- 1) Qual a importância dos resistores de pull-up?

- 2) É possível utilizar um resistor de pull-down no ATmega328p? Quando seria necessário utilizar um?

- 3) Por que utilizamos o registrador DDRx? Pesquise no manual (datasheet) como é feita a configuração lógica em nível de hardware a partir da informação aplicada neste registrador.